به نام خدا

آزمون پایان‌ترم دوم درس ساختمان داده‌ها و الگوریتم‌ها ــ پاییز ۹۹-۹۸ ــ مدرس: مهدی صفرنژاد

آزمون دارای ۴ سؤال است که سؤال ۱ دارای ۷ قسمت است.

۱. سؤال‌های جواب کوتاه. برای هر کدام از این سؤال‌ها درست یا غلط بودن جمله نوشته شده را مشخص کنید و دلیل آن را به اختصار توضیح دهید. می‌تواند جواب را در مستطیل زیر سؤال بنویسید.

الف Using an AVL tree as Dijkstra's priority queue results in the same asymptotic running time as using a Fibonacci heap, when running Dijkstra's algorithm to compute single source shortest paths on a graph with n vertices and $O(n)$ edges.

ب Given a connected weighted directed graph having positive integer edge weights, where each edge weight is at most k, we can compute single source shortest paths in $O(k|E|)$ time.

پ While running Bellman-Ford from vertex s in a simple graph, suppose distance estimate $d'[s, v]$ is assigned a new smaller value D in the kth round of relaxation. Then D is the weight of a path from s to v traversing exactly k edges.

ت

Depth-first search solves single-source shortest paths in an unweighted, directed graph $G = (V, E)$ in $O(|V| + |E|)$-time.

ث

Given a weighted connected undirected graph $G = (V, E)$ containing exactly $|V| - 1$ edges, one can solve weighted Single-Source Shortest Paths from any $s \in V$ in $O(|V|)$ time.

ج

Given a weighted simple directed graph $G = (V, E)$, where every vertex connects to at least $|V|/2$ edges and every edge has strictly negative weight, one can find a maximum-weight directed path from vertex $s \in V$ to $t \in V$ in linear time.

چ

Checking if a string of length k exists in a hash table takes worst-case $O(k)$ time.

در سؤال‌های ۲ تا ۴ باید یک الگوریتم مناسب طراحی کنید. الگوریتم خود را به خوبی توضیح دهید. نیاز به نوشتن اثبات بسیار دقیق نیست ولی دلیل درستی الگوریتم خود را نیز باید به طور مشخص بیان کنید. از الگوریتم‌هایی که در کلاس گفته شده‌اند می‌توانید استفاده کنید و نیاز به اثبات آن‌ها ندارید.

2 A computer program accesses n variables $V = (v_1, \ldots, v_n)$ over time. When a program accesses a variable v, we would like to know the **cache cost** of the access: specifically, the number of **distinct** variables accessed since variable v was last accessed (if v has not been accessed before, its access cost is zero). For example, if a program accesses variables $V = (a, b, c, d)$ in the order (a, b, c, d, b, a, b), then the cache cost of the accesses are:

access #	1	2	3	4	5	6	7
variable	a	b	c	d	b	a	b
cache cost	0	0	0	0	2	3	1

Accesses 1-4 are first accesses so each have cost 0; access 5 has cost 2 because c and d are accessed between the first two accesses of b; access 6 has cost 3 because b, c, and d are accessed between accesses of a; and access 7 has cost 1 because a is accessed between the last two accesses of b.

Describe a dynamic data structure to store variable accesses over time from the start of a program, supporting one operation, `access(`v`)`, which accesses variable v and returns the cost of the access. The operation should run in $O(\log n)$ time where n is the number of variables accessed by the program. State whether your operation running time is worst-case, amortized, and/or expected.

3

- A **blue-labeled** graph is a directed graph where every edge is labeled either blue or not-blue.
- A cycle in a blue-labeled graph is **mostly-blue** if more than half of its edges are blue.
- A vertex is **high-degree** if the sum of its in-degree and out-degree is strictly more than two.

Given a blue-labeled simple graph G containing n vertices and at most $O(n)$ edges, where exactly $k < n$ vertices are high-degree, describe an $O(n + k^3)$-time algorithm to determine whether G contains a mostly-blue cycle.

4 Supervillain Shrunk leads a team of social minions. Each minion has a unique **name**: a string of at most k lowercase English letters. Each pair of minions has at most two **buddy names**, formed by the concatenations of their names in either order (note it is possible for a pair of minions to have only one buddy name if both concatenations are the same). Four minions are **buddy buddies** if a buddy name of two of them is identical to a buddy name of the other two. For example, minions `'alex'`, `'isabel'`, `'alexis'`, and `'abel'` are buddy buddies.

Given a list $A = (a_1, \ldots, a_n)$ containing the names of Shrunk's n minions, describe an $O(kn^2)$-time algorithm to determine whether any four of them are buddy buddies. State whether your algorithm's running time is worst-case, amortized, and/or expected.